

身份认证与知识库问答实现方案

CampusClaw 课程项目实施说明

2026 年 10 月 1 日

本文说明 CampusClaw 当前已经实现的两条核心链路：一是使用随机不透明 Bearer Token 的身份认证，二是限定班级范围、可追溯并可接入大模型的知识库检索与问答。文档同时列出浏览器、Flask API、SQLite、Qdrant 与外部模型服务之间的边界，并说明 Docker 部署时必须保留的配置。

一 实现结论

当前系统已经具备课程演示所需的认证、材料管理、班级隔离、混合检索、切片溯源和问答接口。问答接口支持两种运行方式：默认的离线抽取式回答，以及通过 OpenAI 兼容 Chat Completions 接口调用真实模型。真实模型模式需要在应用容器中配置环境变量；如果 ANSWER_MODE 未设置，服务会有意选择 ExtractiveAnswerProvider，因此回答会呈现固定模板。

能力	当前实现	关键入口
身份认证	随机不透明 Token，服务端只保存 HMAC 摘要	POST /login, Authorization: Bearer <token>
角色权限	教师可上传、修改和删除；学生只读	@role_required("teacher")
班级隔离	班级由 Token 关联用户确定，客户端不能提交 class_id 覆盖	所有材料与检索 SQL
知识检索	keyword、vector、hybrid 三种模式	GET /api/knowledge/search
知识问答	混合检索最多 4 个切片后调用回答 provider	POST /api/ask
来源追溯	回答中的 [1]、[2] 映射为标题、切片、字符区间和原文摘录	citations 字段

二 整体架构

系统将浏览器视为不可信客户端。浏览器只访问 Flask 暴露的业务 API，不直接访问 SQLite、Qdrant、嵌入服务或回答网关。Flask 根据 Bearer Token 查询服务端身份，再以身份关联的 class_id 约束材料列表、

材料详情、下载、检索和问答。SQLite 保存权威原文和切片元数据，Qdrant 只保存向量及班级、材料、切片标识等最小 payload。

层次	职责	是否直接面向浏览器
浏览器工作区	登录、保存当前标签页 Token、发起材料/检索/问答请求、展示出处	是
Flask API	认证、角色校验、班级边界、材料读写、检索编排、回答调用	是
SQLite	用户、Token 摘要、材料、权威原文、切片与索引状态	否
Qdrant	向量集合和按 class_id 的向量过滤	否，仅 Compose 内网
外部嵌入/回答服务	可选的向量生成和大模型生成	否，仅服务端调用

三 身份认证实现

3.1 登录与 Token 签发

客户端向 POST /login 发送 JSON 用户名和密码。服务端从 users 表读取密码哈希，并使用 bcrypt 校验。校验成功后，服务端使用 secrets.token_urlsafe(48) 生成随机不透明 Token，返回 access_token、token_type、expires_in、expires_at 以及用户身份信息。原始 Token 只在登录响应中出现一次。

```
POST /login
Content-Type: application/json

{"username": "student_a1", "password": "student_a1_pass"}

响应字段
access_token 随机不透明访问凭据
token_type   Bearer
expires_in   默认 28800 秒
user_id username role class_id
```

3.2 服务端存储与校验

数据库不保存原始 Token。服务端用 TOKEN_HASH_SECRET 对 Token 计算 HMAC-SHA256 摘要，并在 auth_tokens 表保存 token_hash、user_id、issued_at、expires_at 和 revoked_at。每次受保护请求只接受 Authorization 头中的单个 Bearer Token，随后计算摘要并检查未撤销、未过期和用户存在。Cookie、查询参数、请求体中的 Token 都不会被认证逻辑采用。

请求状态	服务端行为	说明
缺少或无效 Token	HTTP 401, 带 WWW-Authenticate: Bearer	客户端应清除本地 Token 并跳转登录
角色不足	HTTP 403	例如学生调用教师上传接口
跨班材料 ID	HTTP 404	不泄露他班标题、正文或路径
Token 过期或已撤销	HTTP 401	撤销后旧 Token 立即不可用

3.3 网页端使用方式

登录页使用 fetch 发送 JSON 请求，成功后把 access_token 保存到 sessionStorage。工作区的 fetchJson 和下载函数为每次请求自动加入 Authorization: Bearer <token>，并使用 credentials: omit。Token 不写入 Cookie、URL、HTML 或页面正文，因此静态页面壳不会携带身份凭据。登出调用 POST /logout，服务端将当前 auth_tokens 记录写入 revoked_at，浏览器随后清除 sessionStorage。

3.4 认证安全边界

- TOKEN_HASH_SECRET 只注入服务端环境，不能写入镜像、提交 Git 或发送到浏览器。
- 生产部署应使用高熵随机密钥和 HTTPS。更换 TOKEN_HASH_SECRET 会使旧 Token 无法通过摘要校验，需要重新登录。
- Token 有效期由 ACCESS_TOKEN_TTL_SECONDS 控制，默认 8 小时；服务端每次请求检查 expires_at。
- 前端隐藏上传按钮只改善界面，不构成权限控制；上传、修改和删除始终由服务端检查教师角色。

四 教研材料与知识库底座

4.1 材料上传与权威原文

教师通过 POST /api/materials/upload 上传 .txt 或 .md 文件。服务端限制文件大小，要求 UTF-8 编码，并把原文件存入 uploads 目录，把正文写入 knowledge_entries.body_text。正文是权威内容，切片和向量都从它派生，不会覆盖原文。学生调用同一接口会得到 HTTP 403。

4.2 切片与索引

默认 auto 策略按约 800 个字符切片，重叠约 80 个字符，并优先在换行、句号、问号、感叹号或分号等自然断点结束。hierarchy 策略按 Markdown 一级至三级标题分章，过长章节回退到窗口切分；custom 策

略允许显式设置长度、重叠和分隔符。每个切片记录 chunk_index、start_offset、end_offset、offset_basis、strategy、content_hash、index_status 和 vector_status。

存储位置	保存内容	设计目的
knowledge_entries	材料正文和 class_id	作为权威原文
knowledge_chunks	切片正文、序号、偏移、策略、状态	支持检索回表和出处展示
knowledge_fts	SQLite FTS5 索引	支持关键词检索
Qdrant	向量和 class_id、material_id、chunk_id 等标识	支持语义检索，不重复保存正文

五 知识检索实现

5.1 三种检索模式

模式	流程	依赖
keyword	SQLite FTS5 匹配；必要时使用 LIKE 兜底	SQLite
vector	问题生成向量，Qdrant 按 class_id 过滤并按阈值返回，再回 SQLite 取正文	嵌入 provider、Qdrant、SQLite
hybrid	关键词结果和向量结果用 Reciprocal Rank Fusion 合并	以上全部

检索接口为 GET /api/knowledge/search?q=&mode=&limit=，默认 hybrid，limit 上限为 20。服务端先从 Bearer Token 取得用户 class_id，再把 class_id 同时用于 SQLite 查询和 Qdrant filter。即使客户端伪造 class_id，也不能改变服务端检索范围。

5.2 检索结果与溯源字段

每条命中结果包含 snippet 和 source。source 至少包含 material_id、title、chunk_index、start_offset、end_offset、offset_basis、strategy、preview_url 和 download_url。预览和下载地址仍受 Bearer Token、班级范围和材料存在性保护。前端展示切片正文、切片序号和字符区间，便于演示检索为何命中。

六 知识问答实现

6.1 问答请求链路

1. 浏览器向 POST /api/ask 发送 question，可选发送 user/assistant history。客户端不能指定检索班级。
2. 服务端从 Token 取得 class_id，固定使用 hybrid 检索，并最多选取 4 个切片作为证据。
3. 没有命中时直接返回“资料中未找到相关内容”和空 citations，不调用回答服务。
4. 有命中时把标题、切片序号和正文放入回答 provider 的上下文，服务端保留来源顺序。
5. 回答文本经过 citation normalizer，只保留范围内的 [1]、[2] 等引用，并映射回真实 source。
6. 返回 answer 和 citations；前端分别展示回答正文与可追溯出处。

```
POST /api/ask
Authorization: Bearer <access_token>
Content-Type: application/json

{"question": "一次函数的基本形式是什么?"}

响应示例
{
  "answer": "... [1]",
  "citations": [{
    "index": 1,
    "source": {"title": "...", "chunk_index": 0,
      "start_offset": 0, "end_offset": 240},
    "snippet": "..."}
  ]
}
```

6.2 回答 provider

ANSWER_MODE=extractive 时使用离线 ExtractiveAnswerProvider。它用于课程验收和无外网演示，会从第一条证据切片截取正文并附加 [1]，所以不同问题可能出现相似或相同的模板回答。要调用真实 AI，应设置 ANSWER_MODE=openai-compatible。服务端会把问题、最多 4 个证据切片和有限的对话历史翻译为 Chat Completions messages，并使用服务端 API Key 调用配置的 /chat/completions。

配置项	作用	示例或说明
ANSWER_MODE	选择回答模式	openai-compatible 或 extractive
ANSWER_API_BASE_URL	兼容接口基础地址	https://api.deepseek.com，程序追加 /chat/completions
ANSWER_MODEL	服务账号实际可用的模型名	由 API 账户支持情况决定
ANSWER_API_KEY	服务端调用密钥	只放在应用容器环境，不返回浏览器

配置项	作用	示例或说明
ANSWER_TIMEOUT	外部请求超时秒数	可选，当前示例为 30

OpenAI 兼容 provider 要求模型返回标准 `choices[0].message.content`。网络错误、配置不完整、响应结构错误或空回答会返回 HTTP 503，不会静默改用抽取式回答。这样可以明确区分“AI 已接通”和“当前仍使用离线演示模式”。

七 相关源码位置

模块	职责
app/auth.py	登录、Token 签发、HMAC 摘要、Bearer 校验、登出撤销、角色保护
app/materials.py	页面壳、材料列表、详情、下载、教师上传与删除
app/knowledge.py	上传文件类型和 UTF-8 内容解析、材料正文入库
app/chunking.py	auto、custom、hierarchy 切片策略和偏移计算
app/retrieval.py	关键词、向量、混合检索编排和班级范围
app/retrieval_routes.py	检索 API、问答 API、引用映射
app/answer.py	抽取式、通用 HTTP 和 OpenAI 兼容回答 provider
app/vector_store.py	Hash/HTTP 嵌入、内存向量库和 Qdrant 适配器
app/static/login.js	登录后保存 sessionStorage Token
app/static/workspace.js	Authorization 请求、检索/问答界面和出处展示
docker-compose.yml	app、Qdrant、环境变量和持久化卷

结论：认证链路已经是服务端 Bearer Token 方案，知识库链路已经具备本班混合检索和来源追溯。要让演示真正调用 AI，关键不是修改问答接口，而是确保 Ubuntu 运行目录的 `.env` 同时保留 `TOKEN_HASH_SECRET` 并加载 `ANSWER_MODE=openai-compatible` 等配置，然后强制重建 app 容器。